

Bug Fix Log

BITS Digital CodeForge V1.0 – Advanced Grading Console

Submitted by	cybersd12321
Repository	https://github.com/cybersd12321/bits-codeforge-grading-console
Live deployment	https://cybersd12321.github.io/bits-codeforge-grading-console/
Source file	index.html — single file, no build step, one CDN dependency (SheetJS)
Defects resolved	13 total — 10 from the challenge brief, 3 found during a follow-up audit

Section 1 – Bug Fix Log

The ten defects specified in the challenge brief. Each was reproduced before being fixed, and re-tested afterwards. Verification was performed by executing the application inside a minimal DOM harness rather than by inspection alone, so the reproduction and test columns describe observed behaviour.

#	Bug / Issue Identified	How You Reproduced It	Root Cause	Fix Implemented	How You Tested the Fix
1	Modern .xlsx workbooks could not be selected, and parsing corrupted cell values.	Opened the file picker with a .xlsx marks file — it appeared greyed out and could not be chosen. Forcing it through “All files” produced garbled cell values on some browsers.	Two faults on one path. The input carried <code>accept=".xls"</code> , excluding the OOXML extension that every current Excel writes. Parsing used <code>readAsBinaryString()</code> with <code>XLSX.read(data, {type:"binary"})</code> , which relies on a deprecated byte-to-character mapping that silently corrupts bytes.	Changed the input to <code>accept=".xlsx,.xls"</code> . Replaced the reader with <code>readAsArrayBuffer()</code> and <code>XLSX.read(new Uint8Array(buffer), {type:"array"})</code> . Added a missing-library guard and a <code>try/catch</code> that surfaces a readable message instead of failing inside the reader callback.	Pushed generated .xlsx fixtures through the real change handler in a DOM harness: 40 records across 4 courses parsed, headers resolved to BITS ID / Course / Total Marks. A 5-byte garbage file produced a user-facing explanation, not a thrown error.
2	The course dropdown kept the previous file’s courses and repeated every name.	Uploaded file A (3 courses), then file B (2 courses) — the dropdown listed all five. With 24 students in CS F213, that course name appeared 24 times.	<code>data.map(d => d.Course).forEach(c => course.add(...))</code> iterated every row with no de-duplication, and nothing cleared the existing <code><option></code> elements before repopulating.	<code>resetCourseOptions()</code> now runs at the top of the change handler, before the asynchronous read, so a cancelled or failed second upload cannot leave stale courses behind. Population uses <code>[...new Set(records.map(r => r.course))]</code> , sorted with <code>localeCompare</code> .	Uploaded A then B and asserted <code>options.length === courses + 1</code> (placeholder included). Ran 10 consecutive course switches and re-uploads, asserting no accumulation.
3	The Min and Max statistics each displayed the other’s value.	Loaded a course with marks spanning 18–92. The card labelled Min read 92; the card labelled Max read 18.	The markup was cross-wired: <code>Min
<b id="max"> sat above Max
<b id="min"></code> . The code wrote <code>min.textContent = m[0]</code> relying on implicit id-to-global binding, so the lowest mark was written into the node visually labelled Max.	Renamed the value nodes to <code>statMin</code> , <code>statMax</code> , <code>statAvg</code> , <code>statMed</code> and replaced every implicit global with an explicit <code>document.getElementById</code> reference.	Asserted <code>statMin === "18"</code> and <code>statMax === "92"</code> on a real cohort. Added a static check that every <code>getElementById</code> literal in the script resolves to an id present in the markup — the class of mistake that let this hide.

#	Bug / Issue Identified	How You Reproduced It	Root Cause	Fix Implemented	How You Tested the Fix
4	Empty and single-student courses produced NaN and undefined in the statistics panel.	Selected a course with no matching rows: Avg showed NaN and Min showed undefined. Uploading an empty sheet ran the histogram's binning path on an empty array.	<code>m.reduce((a,b)=>a+b,0) / m.length</code> evaluates <code>0/0</code> → NaN on an empty array, and <code>m[0]</code> returns undefined. Neither <code>computeStats()</code> nor <code>drawHistogram()</code> checked length before doing arithmetic.	<code>computeStats()</code> returns null when <code>n === 0</code> and the renderer falls back to an em dash. <code>drawHistogram()</code> paints a labelled empty state and returns before any maths. <code>drawBellCurve()</code> returns early when <code>n < 2</code> , since one point has no distribution.	<code>computeStats([]) === null</code> . A real one-student course (66 marks) asserted <code>min = max = avg = median = 66</code> . Drove empty, header-only and single-student uploads end to end with zero console errors.
5	The median concatenated strings instead of averaging them.	A four-student course whose marks arrived as the strings "79", "81", "60", "40" reported a median in the thousands rather than 69.5.	<code>(m[m.length/2-1] + m[m.length/2]) / 2</code> — with string operands, <code>+</code> is concatenation. "60" + "79" becomes "6079", which is then divided by 2.	<code>const median = n % 2 ? Number(sorted[mid]) : (Number(sorted[mid-1]) + Number(sorted[mid])) / 2</code> , with the array numerically coerced and sorted before the split.	Asserted the median of ["79", "81", "60", "40"] is exactly 69.5. Cross-checked against real data: the 24-student CS F213 cohort returns 63, and the borderline pair (78, 79) returns 78.5.
6	Fractional marks were graded as fractions rather than rounded.	A sheet containing 80.2 carried that value into binning, cutoff comparison and the exported CSV, so a mark could fall inconsistently around an integer boundary.	No normalisation existed between <code>sheet_to_json</code> and the grading logic. Whatever Excel held was used verbatim.	<code>Math.round()</code> is applied once, at the parse boundary, together with clamping into 0–100. Histogram, statistics, cutoff comparison and CSV therefore all see whole marks. Rounding is to the nearest whole integer, per the BITS grading instruction.	Asserted <code>80.2</code> → 80, <code>79.6</code> → 80, <code>49.5</code> → 50, and that every stored mark satisfies <code>Number.isInteger(m) && m >= 0 && m <= 100</code> . Verified through a real workbook, including a number-formatted cell.
7	The normal curve sat left of the bars, and identical marks produced Infinity.	With three students all scoring 55, <code>std</code> evaluated to 0 and the curve height became <code>1/(0 × √2π)</code> — Infinity. On any normal cohort the curve's peak visibly sat left of the tallest bar.	No zero-variance guard existed. The x mapping was <code>px = 30 + (x/10) * 30</code> , a hardcoded 30px step, while bars were laid out at <code>30 + i*32</code> with width 24 — placing true bin centres at <code>42 + i*32</code> . The two drifted 2px per bin, 18px by the last.	Added a guard: non-finite or zero <code>std</code> returns before the division and instead draws a dashed marker at the mean, so identical marks still communicate something. Introduced a single <code>markToX()</code> derived from shared chart constants, so bars and curve read the same geometry and cannot drift again.	Asserted <code>markToX(i*10 + 5)</code> equals the bar centre <code>30 + i*32 + 12</code> for all ten bins, within floating-point tolerance. Confirmed the zero-variance course genuinely yields <code>std === 0</code> and renders with no Infinity or NaN.
8	Reset raised two dialogs in a row, and the hover and pulse animations never appeared.	Clicking Reset Range showed a confirmation; dismissing it immediately raised a second, near-identical one. Changing a grade dropdown produced no visible lift, and changed summary pills never pulsed.	Two sequential <code>confirm()</code> guards sat in the same handler. The animation classes were removed by <code>setTimeout(..., 30)</code> — 30ms — while the corresponding CSS transitions run for .25s, so the class was gone long before the transition could render.	Reduced to a single, clearly worded confirmation. Introduced <code>ANIM_MS = 260</code> so the class always outlives the 250ms transition, applied to the card lift and the pill pulse alike.	Asserted exactly one <code>confirm()</code> call inside the reset handler and that no <code>setTimeout(..., 30)</code> remains anywhere. Drove both the accept and the decline path — declining leaves edited cutoffs untouched.

#	Bug / Issue Identified	How You Reproduced It	Root Cause	Fix Implemented	How You Tested the Fix
9	The grading timer measured how long the browser tab had been open.	Opened the page, left it five minutes, then entered a name and selected a course. Elapsed time already read 05:00, and the finalize message reported that as grading time.	<code>const gradingStartTime = Date.now()</code> executed at module scope, and <code>startTimer()</code> was invoked from a <code>DOMContentLoaded</code> listener, so the clock began at page load regardless of whether grading had started. The display also sat blank for a full second awaiting the first interval tick.	<code>gradingStartTime</code> is initialised to <code>null</code> . <code>startGradingTimer()</code> is called from exactly one place — the course-selection handler, after the instructor-name check passes — so the clock starts only when an instructor and a valid course both exist. It paints immediately, then ticks.	Asserted no interval is scheduled at boot and the clock reads <code>00:00</code> ; that <code>startGradingTimer()</code> has exactly one call site, inside <code>courseEl.onChange</code> ; and that no <code>DOMContentLoaded</code> handler starts it. Simulated 7 min 23 sec and confirmed the reported figure matched.
10	The exported CSV was malformed and opened incorrectly in Excel.	An instructor named Dr. Rao, Meera split across two columns. A metadata row read <code>Course, CS F213</code> with a leading space inside the value. Non-ASCII characters appeared as mojibake when opened in Excel.	The CSV was assembled by string concatenation — <code>Course, \${course.value}</code> — with a stray space baked into the template, no RFC 4180 quoting of any field, and no UTF-8 byte-order mark for Excel to detect the encoding.	A <code>csvField()</code> helper quotes any field containing a comma, double quote, CR, LF or edge whitespace, and escapes inner quotes by doubling them. Rows are joined with CRLF per RFC 4180, <code>\uFEFF</code> is prepended, and the blob is typed <code>text/csv; charset=utf-8</code> .	Captured the generated blob and asserted: the BOM is present; no bare <code>\n</code> exists; <code>Instructor, "Dr. Rao, Meera"</code> is quoted correctly; <code>Course, CS F213</code> has no stray space; and IDs containing commas and quotes round-trip byte-exactly.

Section 2 – Additional Defects Found During Audit

Three further faults surfaced when the completed application was executed scenario by scenario rather than only read. They fall outside the original brief and are listed separately for that reason.

#	Bug / Issue Identified	How You Reproduced It	Root Cause	Fix Implemented	How You Tested the Fix
11	An empty workbook reported success.	Uploaded an empty sheet. The health banner showed a calm green “File verified: 0 valid student records across 0 courses.” while the red line directly beneath it said nothing was gradable — two contradictory statements about the same file.	The banner’s tone was derived only from the presence of anomalies. An empty file has none, so it fell through to the all-clear branch. Zero records was never itself treated as a condition worth reporting.	Zero kept records now forces the warning tone regardless of anomaly count, with a message that says plainly what happened: “No student rows found on that workbook’s first sheet.”	Uploaded an empty sheet and a header-only sheet; asserted the banner carries the warn tone, states that nothing was found, and does not repeat itself in a chip.
12	The CSV carried no record of how long grading took.	Finalized a course, then opened the CSV. It held the instructor and course but no timing information — the elapsed figure existed only in an on-screen message that disappears with the page.	minT and secT were computed in the download handler but interpolated solely into the confirmation text. Nothing wrote them to the file.	The export now opens with a self-describing metadata block: instructor, course, students graded, a sortable finalize timestamp, grading time, and the attempt number.	Drove two finalizes against a controlled clock: the first recorded 7 min 23 sec, the second a cumulative 11 min 5 sec with the attempt counter advancing, and the on-screen message agreed with the file both times.
13	Numeric BITS IDs lost their leading zeros.	A sheet where 0012345 was stored as a number carrying a 0000000 display format exported as 12345.	sheet_to_json with raw values returns the underlying number, 12345. The leading zeros existed only in the cell’s display format, which the raw read discards.	The sheet is read a second time with raw: false, and the formatted text is preferred for the BITS ID — what the instructor sees in the cell is what gets exported. Guarded on row-count alignment between the two passes, falling back to raw values if they diverge.	Built a workbook with an explicitly number-formatted ID cell and confirmed 12345 now exports as 0012345, while alphanumeric, large numeric, and comma- or quote-bearing IDs remain byte-exact.

Verification Summary

- The final review pass recorded 136 automated checks passing with zero failures.
- Scoping: strict mode throughout, zero var declarations, zero accidental globals (verified by diffing the execution context against the declared top-level functions).
- Zero uncaught errors across an empty sheet, a header-only sheet, a 5-byte garbage file, a single-student course, a zero-variance course, ten consecutive course switches, re-uploading mid-session, and resetting cutoffs while a filter is active.
- Cutoff integrity: all 707 possible minimum edits and all 707 maximum edits keep the grade scale continuous; across 200 random configurations every mark from 0 to 100 maps to exactly one band.
- Export fidelity: the generated CSV was verified byte-for-byte, including BOM, CRLF line endings, RFC 4180 quoting and exact BITS ID round-tripping.
- Live deployment: 28 transport and content checks pass against the public URL, including byte-identical delivery of index.html and HTTPS enforcement.

Accessibility note: the checks above cover keyboard operation, focus visibility, colour contrast, semantic table structure and live-region announcements. Full WCAG conformance additionally requires manual testing with assistive technology and expert review, which has not been performed.